
DSC 40B - Homework 05

Due: Wednesday, May 6

Write your solutions to the following problems by handwriting them on another piece of paper. Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer. Homeworks are due via Gradescope at 11:59 p.m.

Problem 1.

For each of the sequences of numbers shown below, insert the numbers into a binary search tree (BST) in the exact order given. Draw the binary tree that results (each node in the tree should be labeled by the number it contains).

a) 50, 30, 70, 20, 40, 60, 80, 10, 35, 45, 55, 65, 75, 85, 5

b) 5, 10, 20, 30, 35, 40, 50, 55, 45, 60, 70, 80, 85, 75, 65

Problem 2.

Suppose a binary search tree has been augmented so that each node contains an additional attribute called **size** which contains the number of nodes in the subtree rooted at that node. Complete the following code so that it computes the value of the k th smallest key in the subtree rooted at **node**, where $k = 1$ is the minimum.

```
def order_statistic(node, k):
    if node.left is None:
        left_size = 0
    else:
        left_size = node.left.size

    order = left_size + 1

    if order == k:
        return node.key
    elif order < k:
        return order_statistic(...)
    else:
        return order_statistic(...)
```

Programming Problem 1.

Recall that a *mode* of a collection is an element which occurs with the greatest frequency. For example, 4 is a mode of the collection 4, 5, 8, 3, 4, 2, 4, 5, 5, -2. 5 is also a mode, since it occurs just as frequently as 4.

In a file named `mode.py`, create a function named `mode(numbers)` which takes in an array of numbers and returns a mode. To receive full credit, your function should have the best possible average case time complexity. If there are multiple modes, your function need only return one of them. You should *not* assume that the numbers are integers or that they are positive.

Note: it's up to you to determine the best possible time complexity and convince yourself that your code is efficient! Think back to when we talked about theoretical lower bounds. Can you come up with one for this problem? This is a situation you'll find yourself in often when writing code: there's usually no resource that

tells you what the best time complexity is for a given problem, so you'll have to convince yourself that your solution is optimal.